

User Guide

Contents

Overview.....	4
Install Python SDK Library.....	4
rApp Software Development Kit.....	5
Service Management and Exposure Package.....	5
Register rApp.....	6
Deregister rApp.....	7
Update Web UI Link of rApp.....	9
Service Registration.....	11
Service Deregistration.....	13
Service Update.....	15
Service Discovery.....	17
Subscribe Service.....	19
Unsubscribe Service.....	20
Data Management and Exposure Package.....	22
Register Data Type.....	23
Deregister Data Type.....	27
Subscribe Data Type Change.....	27
Unsubscribe Data Type Change.....	28
Discover Data Type.....	28
Subscribe Data.....	29
Pull Data.....	32
Push Data.....	33
Request Data.....	33
Configuration Management and Exposure Package.....	34
Read Configuration.....	35
Write Configuration.....	36
Query Alarm Info.....	37
Query Performance Info.....	37
A1 Related Services (A1 Termination) Package.....	38
Query Policy Identifiers.....	39
Query Policies.....	39
Create Policies.....	40
Update Policies.....	42
Query Policy Status.....	43
Subscribe To Policy Status Change.....	44
Delete Policy.....	45
Query Policy Type Identifiers.....	46
Subscribe To Policy Type Availability Change.....	46
Query Policy Type.....	47
Query NearRT-RIC Identifiers.....	48
Schema.....	49
rAppRegister.....	49
appTypeEnum.....	50
serviceRegister.....	50
httpMethodEnum.....	51

accessMethodsEnum.....	52
serviceUpdate.....	52
subscribeRequest.....	53
unsubscribeService.....	54
dataProducerLimits.....	55
accessMethodEnum.....	55
pushPull.....	56
status.....	56
transferMethod.....	56
subscribeDataTypeRequest.....	57
discoverDataType.....	57
dataTypeRegister.....	58
subscribeDataRequest.....	60
pullDataRequest.....	61
pushDataRequest.....	61
requestDataRequest.....	62
WriteConfigModel.....	63
ReadConfigModelV2.....	64
ReadConfigModel.....	65
WriteConfigModelV2.....	65
WriteConfigUrlParamModel.....	66
AlarmDataRequestModel.....	67
PmDataRequestModel.....	68
Policy.....	69
PolicyIds.....	69
PolicyTypeIds.....	70
PolicyStatusEnum.....	70
Exceptions.....	70

Glossary.....	71
----------------------	-----------

Index.....	72
-------------------	-----------

Overview

Python® Software Development Kit (SDK) is a Python-based software package that can be imported into any Non Real Time-RAN Intelligent Controller (NonRT-RIC) Application (rApp) code. This package provides an easy way of invoking R1 Application Programming Interface (API) to the Fujitsu NonRT-RIC. With the Python SDK, the rApp developer can focus on the core of the rApp business logic without worrying regarding the details of interfacing with NonRT-RIC.

The rApp SDK provides ready-to-use functions that wrap the R1 APIs to Service Management and Exposure (SME) and Data Management and Exposure (DME) microservices within the NonRT-RIC. It is integrated with keycloak for user authentication to perform token management, agnostic to rApp core functions.

The rApp SDK performs periodic heartbeat with NonRT-RIC to maintain the active state of the rApp with NonRT-RIC. It has a host of other rApp developer-friendly functions.

The Python SDK provides wrapper functions for the following group of APIs:

- R1-SME: This group of functions enables rApp to register/deregister with the NonRT-RIC, register/deregister, discover, and subscribe the services.
- R1-DME: This group of functions enables rApp to register/deregister data types and send or receive the data.
- rApp SDK:
 - Provides functions to fetch historical data based on the start and end time of the generated data.
 - Provides O1-related functions to fetch performance, fault, and configuration data.
 - Performs token generation for authentication and automatic token regeneration on expiry.
 - Performs the heartbeat function with NonRT-RIC to maintain the active state of the rApp with NonRT-RIC.

Install Python SDK Library

This procedure describes how to install the Python SDK library.

1. Download Python SDK library from the Fujitsu *Partners* website, <https://partners.fnc.fujitsu.com/selfservice> using user credentials (Partners account required).

2. Open terminal command prompt.

3. Change the directory.

Command

```
$ cd rapp-sdk
```

4. Install the pip to upgrade setuptools wheel.

Command

```
$ pip install --upgrade setuptools wheel
```

5. Install the Python setup.

Command

```
$ python setup.py install
```

The Python setup is installed.

6. Clean up the repository.

Command

```
$ cd
$ rm -r rapp-sdk/ -f
```

The repository is removed.

rApp Software Development Kit

- [Service Management and Exposure Package](#) on page 5
- [Data Management and Exposure Package](#) on page 22
- [Configuration Management and Exposure Package](#) on page 34
- [A1 Related Services \(A1 Termination\) Package](#) on page 38
- [Schema](#) on page 49
- [Exceptions](#) on page 70

Service Management and Exposure Package

The following methods of the SDK invokes some of the APIs exposed by the SME.



Note: Before using SME SDK functions, the SME client instance must be created from the SME client class provided by the rApp SDK by adding the following codes in the Python file.

Command

```
from rappsdk.sme.sme_client import SMEClient
sme_client_instance = SMEClient(sme_url, sme_heartbeat_url, username,
password)
```

Table 1: Input Parameters

Key	Mandatory/Optional	Type	Description
sme_url	Mandatory	string	SME Ingress
sme_heartbeat_url	Mandatory	string	SME Heartbeat Ingress
username	Mandatory	string	Username registered in keycloak.
password	Mandatory	string	Password

- [Register rApp](#) on page 6
- [Deregister rApp](#) on page 7
- [Update Web UI Link of rApp](#) on page 9
- [Service Registration](#) on page 11
- [Service Deregistration](#) on page 13
- [Service Update](#) on page 15
- [Service Discovery](#) on page 17

- [Subscribe Service](#) on page 19
- [Unsubscribe Service](#) on page 20

Register rApp

Method Name

```
register_rapp(self, rAppDetails)
```

Description

This method invokes SME API for registering an rApp with NonRT-RIC framework. This method takes an instance of `rAppRegister` class as an input and returns information registered with SME as a dictionary.



Note:

- SME has a running pod monitor that monitors rApp deployments running in K8s namespaces.
- Verify the name of the `rApp` and `rAppVersion` are the same as the pod deployment name and image tag; otherwise, the pod monitor cannot find the rApp deployment in the monitoring process and ends up deregistering and deleting the rApp.

Table 2: Input Parameters

Key	Mandatory/Optional	Type	Description
<code>rAppDetails</code>	Mandatory	rAppRegister on page 49	<code>rAppRegister</code> instance containing the rApp details needed for registration with SME.

Table 3: Returns

Status	Description	Type	Result
Successful	rApp is successfully registered for the first time.	dictionary	Response <pre>{ "rAppName": "string", "vendor": "string", "rAppVersion": "string", "certificates": {}, "appType": "string", "interfaceEndpoint": "string", "webUILink": "string", "rAppId": "string", "listOfServices": [], "heartbeatPeriod": "60", "rAppStatus": "string" }</pre>
	Trying to register an rApp that is already registered (same <code>rAppName</code> and <code>rAppVersion</code>).	dictionary	Response <pre>{ "rAppId": "<existing rApp Id>" }</pre>

Status	Description	Type	Result
Unsuccessful	Failure in registration of rApp.	Exceptions on page 70	Method raises an instance of exception if the operation is unsuccessful.

Table 4: Parameters

Key	Type	Description
rAppName	string	Name of rApp provided in input.
vendor	string	Vendor name
rAppVersion	string	Version of rApp.
certificates	dictionary	Currently, dictionary is empty.
appType	appTypeEnum on page 50	Type of rApp
interfaceEndpoint	string	Uniform Resource Locator (URL) or endpoint of rApp.
webUILink	string	Web User Interface (UI) link of the rApp.
rAppId	string	Random string generated by SME to identify every unique rApp.
listOfServices	list[string]	List of strings that contains services registered by rApp. It is empty at the time of new registration.
heartbeatPeriod	string	Period in which rApp has to send health check requests to SME after every fixed time interval which is given in seconds.
rAppStatus	string	Status of an rApp.

Deregister rApp

Method Name

```
deregister_rapp(self, rAppId)
```

Description

This method invokes SME API for deregistering the rApp from the NonRT-RIC framework. This method takes rAppId as input and returns information of the deregistered rApp as a dictionary.

Notes (Deregistration of rApp Effects):

- rApp status changed to `INACTIVE`.
- Clears the subscriptions made by rApp to the services in SME.
- Unsubscribes from `dataTypeId` type change subscriptions and data subscriptions in DME.
- Deregisters the services that rApp has registered in SME.
- Deregisters the `dataTypeIds` that rApp has registered in DME and delete the consumer records associated with the `dataTypeIds`.
- Deletes the policies that rApp has created in A1 termination.

Table 5: Input Parameters

Key	Mandatory/Optional	Type	Description
<code>rAppId</code>	Mandatory	string	Id of the rApp which must be deregistered from the NonRT-RIC framework.

Table 6: Returns

Status	Description	Type	Result
Successful	rApp is successfully deregistered.	dictionary	Response <pre> { "rAppName": "string", "vendor": "string", "rAppVersion": "string", "certificates": {}, "appType": "string", "interfaceEndpoint": "string", "webUILink": "string", "rAppId": "string", "listOfServices": [], "heartbeatPeriod": "60", "rAppStatus": "ACTIVE" } </pre>
Unsuccessful	Failure in deregistration of rApp.	Exceptions on page 70	Method raises an instance of exception if the operation is unsuccessful.

Table 7: Parameters

Key	Type	Description
<code>rAppName</code>	string	Name of rApp provided in input.
<code>vendor</code>	string	Vendor name
<code>rAppVersion</code>	string	Version of rApp
<code>certificates</code>	dictionary	Currently, dictionary is empty.
<code>appType</code>	appTypeEnum on page 50	Type of rApp

Key	Type	Description
interfaceEndpoint	string	URL or endpoint of rApp.
webUILink	string	Web UI link of the rApp.
rAppId	string	Random string generated by SME to identify every unique rApp.
listOfServices	list[string]	List of strings that contains services registered by rApp. It is empty at the time of new registration.
heartbeatPeriod	string	Period in which rApp has to send health check requests to SME after every fixed time interval which is given in seconds.
rAppStatus	string	Status of the rApp

Update Web UI Link of rApp

Method Name

```
update_rapp_webuilink(self, rAppId, webUILink)
```

Description

This method invokes SME API for updating the web UI link of an rApp registered with the NonRT-RIC framework. This method takes the rAppId of the rApp, and the web UI link has to be updated for the rApp of the provided rAppId.

Table 8: Input Parameters

Key	Mandatory/Optional	Type	Description
rAppId	Mandatory	string	ID of the rApp whose web UI link has to be updated.
webUILink	Mandatory	string	webUILink that has to be updated for the rApp.

Table 9: Returns

Status	Description	Type	Result
Successful	webUILink for the rApp has been updated.	dictionary	Response <pre> { "rAppName": "string", "vendor": "string", "rAppVersion": "string", "certificates": {}, "appType": "string", "interfaceEndpoint": "string", "webUILink": "string", "rAppId": "string", "listOfServices": [], "heartbeatPeriod": "60", "rAppStatus": "ACTIVE" } </pre>
Unsuccessful	Failure in updating the webUILink of rApp.	Exceptions on page 70	Method raises an instance of exception if the operation is unsuccessful.

Table 10: Parameters

Key	Type	Description
rAppName	string	Name of rApp provided in input.
vendor	string	Vendor name
rAppVersion	string	Version of rApp
certificates	dictionary	Currently, dictionary is empty.
appType	appTypeEnum on page 50	Type of rApp
interfaceEndpoint	string	URL or endpoint of rApp.
webUILink	string	Web UI link of the rApp.
rAppId	string	Random string generated by SME to identify every unique rApp.
listOfServices	list[string]	List of strings that contains services registered by rApp. It is empty at the time of new registration.
heartbeatPeriod	string	Period in which rApp has to send health check requests to SME after every fixed time interval which is given in seconds.

Key	Type	Description
rAppStatus	string	Status of the rApp.

Service Registration

Method Name

```
register_service(self, serviceDetails)
```

Description

This method invokes SME API for registering a service with the NonRT-RIC framework. This method takes an instance of `serviceDetails` class as an input and returns service details of the registered service as a dictionary.

Table 11: Input Parameters

Key	Mandatory/Optional	Type	Description
serviceDetails	Mandatory	serviceRegister on page 50	serviceRegister instance containing the service details needed for registration with SME.

Table 12: Returns

Status	Description	Type	Result
Successful	Service has been successfully registered by rApp.	dictionary	Response <pre> { "rAppId": "rapp_ID", "serviceName": "service_name", "serviceVersion": "string", "accessMethod": "RESTApi", "communicationMechanism": {}, "serviceEndpoint": "string", "httpMethod": "string", "serviceLimitations": {}, "serviceCapabilities": {}, "serviceId": "service_ID", "serviceStatus": "ACTIVE" } </pre>
	Trying to register a service that is already registered (same serviceName and serviceVersion).	dictionary	Response <pre> { "rAppId": "rapp_ID", "serviceName": "service_name", "serviceVersion": "string", "accessMethod": "RESTApi", "communicationMechanism": {}, "serviceEndpoint": "string", "httpMethod": "string", "serviceLimitations": {}, "serviceCapabilities": {}, "serviceId": "service_ID", "serviceStatus": "ACTIVE" } </pre>
Unsuccessful 1	Failure in registration of service.	Exceptions on page 70	Method raises an instance of exception if the operation is unsuccessful.

Table 13: Parameters

Key	Type	Description
rAppId	string	ID of the rApp whose service must be registered.
serviceName	string	Name of the service to be registered.
serviceVersion	string	Version of the service.
accessMethod	accessMethodsEnum on page 52	Indicates the access type for the given service.
communicationMechanism	dictionary	Currently, dictionary is empty.
serviceEndpoint	string	URL or endpoint of service.
httpMethod	httpMethodEnum on page 51	HTTP method to invoke the service endpoint.
serviceLimitations	dictionary	Currently, dictionary is empty.
serviceCapabilities	dictionary	Currently, dictionary is empty.
serviceId	string	Random string generated by SME to identify every unique service.
serviceStatus	string	serviceStatus can be ACTIVE or INACTIVE. It is ACTIVE for new service registration.

Service Deregistration

Method Name

```
deregister_service(self, rAppId, serviceId)
```

Description

This method invokes SME API for deregistering the service associated with the rAppId from the NonRT-RIC framework. This method takes rAppId and serviceId as input and returns the service details of the deregistered service as a dictionary.

Table 14: Input Parameters

Key	Mandatory/Optional	Type	Description
rAppId	Mandatory	string	ID of the rApp whose service must be deregistered from the NonRT-RIC framework.

Key	Mandatory/Optional	Type	Description
<code>serviceId</code>	Mandatory	string	ID of the service which must be deregistered.

Table 15: Returns

Status	Description	Type	Result
Successful	Service has been successfully deregistered.	dictionary	Response <pre>{ "rAppId": "rApp_ID", "serviceName": "service_name", "serviceVersion": "string", "accessMethod": "RESTApi", "communicationMechanism": {}, "serviceEndpoint": "string", "httpMethod": "string", "serviceLimitations": {}, "serviceCapabilities": {}, "serviceId": "service_ID", "serviceStatus": "ACTIVE" }</pre>
Unsuccessful	Failure in deregistration of service	Exceptions on page 70	Method raises an instance of exception if the operation is unsuccessful.

Table 16: Parameters

Key	Type	Description
<code>rAppId</code>	string	ID of the rApp whose service must be deregistered.
<code>serviceName</code>	string	Name of the service to be deregistered.
<code>serviceVersion</code>	string	Version of the service.
<code>accessMethod</code>	accessMethodsEnum on page 52	Indicates the access type for the given service.
<code>communicationMechanism</code>	dictionary	Currently, dictionary is empty.
<code>serviceEndpoint</code>	string	URL or endpoint of service.
<code>httpMethod</code>	httpMethodEnum on page 51	HTTP method to invoke the service endpoint.
<code>serviceLimitations</code>	dictionary	Currently, dictionary is empty.

Key	Type	Description
<code>serviceCapabilities</code>	dictionary	Currently, dictionary is empty.
<code>serviceId</code>	list[string]	Random string generated by SMEs to identify every unique service.
<code>serviceStatus</code>	string	<code>serviceStatus</code> can be ACTIVE or INACTIVE. It is ACTIVE for deregistered services.

Service Update

Method Name

```
update_service(self, rAppId, serviceId, serviceDelta)
```

Description

This method invokes SME API for updating service details associated with an rApp. This method takes `rAppId`, `serviceId`, and `serviceDelta` as input and returns the updated service details of the given service as a dictionary.

Table 17: Input Parameters

Key	Mandatory/Optional	Type	Description
<code>rAppId</code>	Mandatory	string	ID of the rApp whose associated service details have to be updated.
<code>serviceId</code>	Mandatory	string	ID of the service whose details have to be updated.
<code>serviceDelta</code>	Mandatory	serviceUpdate on page 52	<code>serviceUpdate</code> instance containing the details of the service to be updated.

Table 18: Returns

Status	Description	Type	Result
Successful	Service details are successfully updated by rApp.	dictionary	Response <pre> { "rAppId": "rapp_ID", "serviceName": "service_name", "serviceVersion": "string", "accessMethod": "RESTApi", "communicationMechanism" : {}, "serviceEndpoint": "string", "httpMethod": "string", "serviceLimitations": {}, "serviceCapabilities": {}, "serviceId": "service_ID", "serviceStatus": "ACTIVE" } </pre>
	Trying to update the details of the service to the rApp that is already registered (same serviceName and serviceVersion).	dictionary	Response <pre> { "rAppId": "rapp_ID", "serviceName": "service_name", "serviceVersion": "string", "accessMethod": "RESTApi", "communicationMechanism" : {}, "serviceEndpoint": "string", "httpMethod": "string", "serviceLimitations": {}, "serviceCapabilities": {}, "serviceId": "service_ID", "serviceStatus": "ACTIVE" } </pre>

Status	Description	Type	Result
Unsuccessful	Failure in registration of service.	Exceptions on page 70	Method raises an instance of exception if the operation is unsuccessful.

Table 19: Parameters

Key	Type	Description
rAppId	string	ID of the rApp whose service must be updated.
serviceName	string	Name of the service to be updated.
serviceVersion	string	Version of the service.
accessMethod	accessMethodsEnum on page 52	Indicates the access type for the given service.
communicationMechanism	dictionary	Currently, dictionary is empty.
serviceEndpoint	string	URL or endpoint of service.
httpMethod	httpMethodEnum on page 51	HTTP method to invoke the service endpoint.
serviceLimitations	dictionary	Currently, dictionary is empty.
serviceCapabilities	dictionary	Currently, dictionary is empty.
serviceId	string	Random string generated by SMEs to identify every unique service.
serviceStatus	string	serviceStatus can be ACTIVE or INACTIVE. It is ACTIVE for existing active services.

Service Discovery

Method Name

```
discover_services(self, rAppId, serviceName, serviceVersion)
```

Description

This method invokes SME API for discovering the services registered by rApp (excluding the services registered by rApp that is doing the discovery) and provide the details. If a service name is given and the service is registered in that name, it only returns the details of that specific service. If the service version is given and a service is registered in that name and version, it only returns the details of that specific version of the service.

This method takes rAppId, serviceName (optional), and serviceVersion (optional) as input and returns the serviceDetails as a list of the dictionary.

Table 20: Parameters

Key	Mandatory/Optional	Type	Description
<code>rAppId</code>	Mandatory	string	ID of the rApp that has to discover the services.
<code>serviceName</code>	Optional	string	Name of the service whose details have to be returned. If none, all the services registered with the rApp is returned.
<code>serviceVersion</code>	Optional	string	Version of the service whose details have to be returned. If none, all the services registered with the provided <code>serviceName</code> , if found, are returned.

Table 21: Returns

Status	Description	Type	Result
Successful	Service is successfully discovered.	list[dictionary]	Response <pre>[{ "serviceName": "string", "serviceVersion": "string", "accessMethod": "string", "communicationMechanism": {}, "serviceEndpoint": "string", "httpMethod": "string" "serviceLimitations": {}, "serviceCapabilities": {}, "serviceStatus": "string" }]</pre>
	No services found.	dictionary	{ }
	Trying to discover service with an <code>rAppId</code> that does not exist.	dictionary	{ }
	Trying to discover service with <code>rAppId</code> whose status is INACTIVE.	dictionary	{ }
Unsuccessful	Failure in registration of service.	Exceptions on page 70	Method raises an instance of exception if the operation is unsuccessful.

Table 22: Parameters

Key	Type	Description
<code>serviceName</code>	string	Name of the service to be registered.
<code>serviceVersion</code>	string	Version of the service
<code>accessMethod</code>	accessMethodsEnum on page 52	Indicates the access type for the given service.
<code>communicationMechanism</code>	dictionary	Currently, dictionary is empty.
<code>serviceEndpoint</code>	string	URL or endpoint of service.
<code>httpMethod</code>	httpMethodEnum on page 51	HTTP method to invoke the service endpoint.
<code>serviceLimitations</code>	dictionary	Currently, dictionary is empty.
<code>serviceCapabilities</code>	dictionary	Currently, dictionary is empty.
<code>serviceStatus</code>	string	<code>serviceStatus</code> can be ACTIVE or INACTIVE. It is ACTIVE for existing services.

Subscribe Service

Method Name

```
subscribe_services(self, subscribe_service)
```

Description

This method invokes SME API for subscribing to a service registered with the NonRT-RIC framework. This method takes an instance of the `subscribeRequest` class as input and returns the details of the subscription as a dictionary.

Table 23: Parameters

Key	Mandatory/Optional	Type	Description
<code>subscribe_service</code>	Mandatory	subscribeRequest on page 53	<code>subscribeRequest</code> instance containing the details of the service to be subscribed.

Table 24: Returns

Status	Description	Type	Result
Successful	Subscription successful	dictionary	Response <pre>{ "rAppId": "string", "subscribedServices": [< List of dictionaries having details of subscribed services >] }</pre>
Unsuccessful	Failure to subscribe to services by rApp.	Exceptions on page 70	Method raises an instance of exception if the operation is unsuccessful.

Table 25: Parameters

Key	Type	Description
rAppId	string	ID of the rApp which has subscribed to the service.
subscribedServices	list[dictionary]	List of dictionaries having details of services subscribed by the rApp. Example <pre>[{ "_id": "string", "serviceName": "string", "serviceVersion": "string" }, { "_id": "string", "serviceName": "string", "serviceVersion": "string" }]</pre>
_id	string	ID to show the status of service. ACTIVE/ INACTIVE denotes service availability or unavailability.
serviceName	string	Name of the service.
serviceVersion	string	Version of the service.

Unsubscribe Service

Method Name

```
unsubscribe_services(self, unsubscribe_service)
```

Description

This method invokes SME API for unsubscribing from a subscribed service. This method takes an instance of `unSubscribeService` class having details of the service to be unsubscribed as an input and returns the `serviceId` of the unsubscribed service, `rAppId`, and list of remaining subscribed services as a dictionary.

Table 26: Input Parameters

Key	Null	Type	Description
<code>unsubscribe_service</code>	—	unSubscribeService on page 54	<code>unSubscribeService</code> instance containing the details of the service to be unsubscribed.

Table 27: Returns

Status	Description	Type	Result
Successful	Subscription successful	dictionary	Response <pre>{ "rAppId": "string", "subscribedServices": [< List of dictionaries having details of pending service subscriptions by the rApp >] }</pre>
	No pending service subscription by the rApp.	dictionary	Response <pre>{ "rAppId": "string", "subscribedServices": [] }</pre>
Unsuccessful	Failure to subscribe to services by rApp.	Exceptions on page 70	Method raises an instance of exception if the operation is unsuccessful.

Table 28: Parameters

Key	Type	Description
<code>rAppId</code>	string	ID of the rApp that made the subscription.

Key	Type	Description
subscribedServices	list[dictionary]	List of dictionaries having details of pending service subscriptions by the rApp. Example <pre>[{ "_id": "string", "serviceName": "string", "serviceVersion": "string" }, { "_id": "string", "serviceName": "string", "serviceVersion": "string" }]</pre> If no pending service subscriptions exist by the rApp, subscribedServices is an empty list.
_id	string	ID to show the status of service. ACTIVE/ INACTIVE denotes service availability or unavailability.
serviceName	string	Name of the service.
serviceVersion	string	Version of the service.

Data Management and Exposure Package

The following SDK methods use or invoke the APIs exposed by DME.



Note: Before using DME SDK functions, the DME client instance must be created from the DME client class provided by the rApp SDK by adding the following codes in the Python file.

Command

```
from rappsdk.dme.dme_client import DMEClient
dme_client_instance = DMEClient(dme_url=, username=, password=)
```

Table 29: Input Parameters

Key	Null	Type	Description
dme_url	—	string	DME server URL
username	—	string	Registered username (Keycloak credentials)

Key	Null	Type	Description
password	—	string	Password corresponds to the given username (keycloak credentials).

- [Register Data Type](#) on page 23
- [Deregister Data Type](#) on page 27
- [Subscribe Data Type Change](#) on page 27
- [Unsubscribe Data Type Change](#) on page 28
- [Discover Data Type](#) on page 28
- [Subscribe Data](#) on page 29
- [Pull Data](#) on page 32
- [Push Data](#) on page 33
- [Request Data](#) on page 33

Register Data Type

Method name

```
register_dataType(self, dataTypeDetails)
```

Description

This method register a data type to the NonRT-RIC framework. Only a registered rApp with valid credentials can register a data type.

Table 30: Input Parameters

Key	Mandatory/Optional	Type	Description
dataTypeDetails	Mandatory	dataTypeRegister on page 58	dataTypeRegister instance that contains the details of the data type for registration.

Table 31: Returns

Status	Description	Type	Result
Successful	New dataTypeId registration is successful.	dictionary	Response <pre> { "dataTypeId": "string", "rappId": "RAPP_ID", "rappName": "string", "accessMethod": "string", "accessDetails": Dictionary, "interfaceEndpoint": "string", "collectionMethod": "string", "availabilityStatus": "string", "collectionPeriod": integer, "dataSchema": List, "producerLimits": { "maxPushTime": integer, "maxConsumers": integer, "maxTransferRate": integer }, "producerCapabilities": "string" } </pre>
	dataTypeId is already registered.	dictionary	Response <pre> { "dataTypeId": "string", "rappId": "RAPP_ID", "rappName": "string", "accessMethod": "string", "accessDetails": Dictionary, "interfaceEndpoint": "string", "collectionMethod": "string", "availabilityStatus": "string", "collectionPeriod": integer, "dataSchema": List, "producerLimits": { "maxPushTime": integer, "maxConsumers": integer, "maxTransferRate": integer }, "producerCapabilities": "string" } </pre>
Unsuccessful	Failure in registration of dataTypeId by rApp.	Exceptions on page 70	Method raises an instance of exception if the operation is unsuccessful.

Table 32: Parameters

Key	Type	Description
dataTypeId	string	Name of the dataTypeId that has been registered.
rappId	string	ID of the rApp that registered the dataTypeId.

Key	Type	Description
rappName	string	Name of the rApp that registered the dataTypeId.
accessMethod	string	Type of access method through which the data can be produced to the subscribed consumer.
accessDetails	dictionary	When accessMethod is set to MsgBus (Message Bus), the accessDetails dictionary includes the following fields: • url has the Kafka[®] host+ port. • topic has the name of the topic from or to which the data can be consumed or produced. It is same as the name of the dataTypeId. Example <pre>"accessDetails": { "url": < kafka host: port > , "topic": < name of topic > }</pre> When accessMethod is set to RESTApi, the accessDetails dictionary includes the following fields: • url has the name of the dataTypeId that is registered. Example <pre>"accessDetails": { "url": < name of dataTypeId > , }</pre>

Key	Type	Description
accessDetails	dictionary	When accessMethod is set to WS (WebSocket), the accessDetails dictionary includes the following fields: • url has the DME WS ingress. • topic has the name of the topic from or to which the data can be consumed or produced. It is same as the name of the dataTypeId. • token has the authentication token that has to be passed in headers for WS ingress when publishing the data. Example <pre>"accessDetails": { "url": < DME websocket ingress > , "topic": < Name of the websocket topic to subscribe to > , "token": < Authentication token required to publish data via websocket > }</pre>
interfaceEndpoint	string	Interface endpoint of the rApp, which is provided to SME when registering rApp.
collectionMethod	string	Desired collection method for the dataTypeId.
availabilityStatus	string	(Not used)
collectionPeriod	integer	(Not used)
dataSchema	list	Schema of the data produced. It is strictly followed when the accessMethod is RESTapi or WS.
maxPushTime	integer	(Not used)
maxConsumers	integer	(Not used)
maxTransferRate	integer	(Not used)
producerCapabilities	string	(Not used)

Deregister Data Type

Method name

```
deregister_dataType(self, dataTypeId)
```

Description

This method deregisters a data type from the NonRT-RIC framework.

Table 33: Input Parameters

Key	Mandatory/Optional	Type	Description
dataTypeId	Mandatory	string	dataTypeId which is meant to be deregistered.

Table 34: Returns

Status	Description	Type	Result
Successful	dataTypeId is successfully deregistered.	dictionary	Empty dictionary after a successful operation.
Unsuccessful	Failure in registration of dataTypeId.	Exceptions on page 70	Method raises an instance of exception if the operation is unsuccessful.

Subscribe Data Type Change

Method Name

```
subscribe_dataType_change(self, dataTypeId, rapp_id)
```

Description

This method subscribe to a data type change from the NonRT-RIC framework to receive the list of active data types.

Table 35: Input Parameters

Key	Mandatory/Optional	Type	Description
data_type_id	Mandatory	String	Data type identifier should be subscribed for its availability notifications. This identifier could be both registered and not registered.
rapp_id	Mandatory	String	ID of the registered rApp.

Table 36: Returns

Status	Description	Type	Result
Successful	Subscribed to dataTypeId change	dictionary	Empty dictionary after a successful operation.

Status	Description	Type	Result
Unsuccessful	Failure in subscription to dataTypeId change	Exceptions on page 70	Method raises an instance of exception if the operation is unsuccessful.

Unsubscribe Data Type Change

Method name

```
unsubscribe_dataType_change(self, data_type_id, rapp_id)
```

Description

This method unsubscribes from a data type change from the NonRT-RIC framework.

Table 37: Input Parameters

Key	Mandatory/Optional	Type	Description
data_type_id	Mandatory	string	Data type identifier should be unsubscribed for its availability notifications.
rapp_id	Mandatory	string	ID of the registered rApp.

Table 38: Returns

Status	Dictionary	Type	Result
Successful	Unsubscribed from dataTypeId change.	dictionary	Empty dictionary after a successful operation.
Unsuccessful	Failure in unsubscription from dataTypeId change.	Exceptions on page 70	Method raises an instance of exception if the operation is unsuccessful.

Discover Data Type

Method name

```
discover_dataType(self, discoverDataTypeDetails)
```

Description

This method discovers all data type IDs, excluding those registered by caller of the API.

Table 39: Input Parameters

Key	Null	Type	Description
discoverDataTypeDetails	—	discoverDataType on page 57	Data model that contains the filter criteria of the searched data types.

Table 40: Returns

Status	Description	Type	Result
Successful	dataTypeId is discovered successfully.	dictionary	Response <pre>{ "data": [< List of string >] }</pre>
Unsuccessful	Failure in discovery of dataTypeId.	Exceptions on page 70	Method raises an instance of exception if the operation is unsuccessful.

Table 41: Parameters

Key	Type	Description
data	list	Includes a list of dataTypeIds that were found in the discovery.

Subscribe Data

Method name

```
subscribe_data(self, subscribeDataDetails)
```

Description

This method subscribes to data registered with the NonRT-RIC framework.

Table 42: Input Parameters

Key	Mandatory/ Optional	Type	Description
subscribeDataDetails	Mandatory	subscribeDataRequest on page 60	Data model that contains the details for subscribing service details.

Table 43: Returns

Status	Description	Type	Result
Successful	Subscribed to data produced by dataTypeId.	dictionary	Response <pre>{ "accessMethod": "string", "accessDetails": < Dictionary with details to provide > , "interfaceEndpoint": "string", "collectionMethod": "string", "collectionPeriod": integer, "scope": "string", "target": "string" }</pre>
	Already subscribed to the same data produced by dataTypeId.	dictionary	Response <pre>{ "accessMethod": "string", "accessDetails": < Dictionary with details to provide > , "interfaceEndpoint": "string" }</pre>
Unsuccessful	Failure in subscription to data produced by dataTypeId.	Exceptions on page 70	Method raises an instance of exception if the operation is unsuccessful.

Table 44: Parameters

Key	Type	Description
accessMethod	string	Type of access method through which the data can be produced to the subscribed consumer.

Key	Type	Description
accessDetails	dictionary	When accessMethod is set to MsgBus, the accessDetails dictionary includes the following fields: • url has the Kafka host+ port. • topic has the name of the topic from or to which the data can be consumed or produced. It is same as the name of the dataTypeId. Example <pre>"accessDetails": { "url": < kafka host: port > , "topic": < name of topic > }</pre> When accessMethod is set to RESTApi, the accessDetails dictionary includes the following fields: • url has the name of the dataTypeId that is registered. Example <pre>"accessDetails": { "url": < name of dataTypeId > , }</pre> When accessMethod is set to WS, the accessDetails dictionary includes the following fields: • url has the DME WS ingress. • topic has the name of the topic from or to which the data can be consumed or produced. It is same as the name of the dataTypeId. • token has the authentication token that has to be passed in headers for WS ingress when publishing the data. Example <pre>"accessDetails": { "url": < DME websocket ingress > , "topic": < Name of the websocket topic to subscribe to > , "token": < Authentication token required to publish data via websocket > }</pre>

Key	Type	Description
<code>interfaceEndpoint</code>	string	Interface endpoint of the rApp.
<code>collectionMethod</code>	string	Desired collection method for the <code>dataTypeId</code> . It can be PUSH or PULL.
<code>collectionPeriod</code>	integer	(Not used)
<code>scope</code>	string	(Not used)
<code>target</code>	string	(Not used)

Pull Data

Method name

```
pullData(self, pullDetails)
```

Description

This method pulls data from a data type registered with the NonRT-RIC framework.

Table 45: Input Parameters

Key	Mandatory/Optional	Type	Description
<code>pullDetails</code>	Mandatory	pullDataRequest on page 61	Data model that contains the payload for pull data API.

Table 46: Returns

Status	Description	Type	Result
Successful	Data pull is successful with matching data.	list[dictionary]	Response <pre>[<List of dictionary where each dictionary is a data record that is pulled>]</pre>
	Using the <code>laguna_ref</code> hash string provided by <code>request_data</code> , which includes querying for a field in the result that does not exist in the data record.	list[dictionary]	[{ }]
Unsuccessful	Failure in pulling the data.	Exceptions on page 70	Method raises an instance of exception if the operation is unsuccessful.

Push Data

Method name

```
pushData(self, pushDetails)
```

Description

This method pushes data to a data type registered with the NonRT-RIC framework.

Table 47: Input Parameters

Key	Mandatory/Optional	Type	Description
pushDetails	Mandatory	pushDataRequest on page 61	Data model that contains the payload for push data API.

Table 48: Returns

Status	Description	Type	Result
Successful	Data is successfully pushed	dictionary	Response <pre>{"lagunaref": "string"}</pre>
Unsuccessful	Failure in pushing data to dataTypeId.	Exceptions on page 70	Method raises an instance of exception if the operation is unsuccessful.

Table 49: Parameters

Key	Type	Description
lagunaref	string	Hash string which references to the pushed data. This hash string can be used to retrieve the pushed data using pull_data API in DME.

Request Data

Method name

```
request_data(self, requestDetails)
```

Description

Request data retrieves historical data of a dataTypeId. It checks against the stored data corresponding to the given dataTypeId in DME and prepares appropriate queries to be used in [Pull Data](#) on page 32.

Table 50: Input Parameters

Key	Mandatory/Optional	Type	Description
requestDetails	Mandatory	Request Data on page 33	Data model that contains the payload for request data API

Table 51: Returns

Status	Description	Type	Result
Successful	Data request is successful.	dictionary	Response <pre>{ "lagunaref": "string" }</pre>
Unsuccessful	Failure in requesting data	Exceptions on page 70	Method raises an instance of exception if the operation is unsuccessful.

Table 52: Parameters

Key	Type	Description
lagunaref	string	Hash string that references to the requested data. This hash string can be used to retrieve the requested data using the <code>pull_data</code> API in DME.

Configuration Management and Exposure Package

The following methods of the SDK support some of the APIs exposed by the Configuration Management and Exposure (CME).



Note: Before using CME SDK functions, the CME client instance must be created from the CME client class provided by the rApp SDK by adding the following codes in the Python file.

Command

```
from rappsdk.cme.cme_client import CMEClient
cme_client_instance = CMEClient(cme_url, username, password)
```

Table 53: Input Parameters

Key	Mandatory/Optional	Type	Description
cme_url	Mandatory	string	CME server URL
username	Mandatory	string	Username registered in keycloak
password	Mandatory	string	Password

- [Read Configuration](#) on page 35
- [Write Configuration](#) on page 36
- [Query Alarm Info](#) on page 37
- [Query Performance Info](#) on page 37

Read Configuration

Method Name

```
read_configuration(self, readConfigurationDetails, version)
```

Description

This method allows rApp to read the configuration of any network device through Virtuora® Mobility Controller (MC) and return its configurations.

This method takes a dictionary as an input, a sample input can be found in the following table.

Table 54: Input Parameters

Key	Mandatory/ Optional	Type	Description
<code>readConfigurationDetails</code>	Mandatory	ReadConfigModel on page 65 or ReadConfigModelV2 on page 64	Data model instance, containing the configuration input needed to read the configuration.
<code>version</code>	Optional	<code>["v1", "v2"]</code>	- If version is not given, it considers v1 as default argument. - If version input is v1, SDK makes API call to <code>read_configuration v1</code> API, which accepts ReadConfigModel on page 65 - If version input is v2, SDK makes API call to <code>read_configuration v2</code> API, which accepts ReadConfigModelV2 on page 64.

Table 55: Returns

Status	Description	Type	Result
Successful	Using <code>read_configuration</code> to get a list of devices.	<code>list[dictionary]</code>	Response <pre>[List of dictionaries where each dictionary includes details of CU & DU devices]</pre>
	Using <code>read_configuration</code> to read CM information of a given Central Unit (CU) or Distributed Unit (DU) device.	<code>dictionary</code>	Response <pre>{CM information of CU or DU device}</pre>

Status	Description	Type	Result
Unsuccessful	Failure in obtaining list of devices or CM information of CU or DU device.	Exceptions on page 70	Method raises an instance of exception if the operation is unsuccessful.

Write Configuration

Method Name

```
write_configuration(self, writeConfigurationDetails, version)
```

Description

This method allows rApp to modify the configuration of any network devices through Virtuora MC.

This method takes a dictionary as an input.

Table 56: Input Parameters

Key	Mandatory/Optional	Type	Description
<code>writeConfigurationDetails</code>	Mandatory	WriteConfigModel on page 63 or WriteConfigModelV2 on page 65	Data model instance, containing the configuration input needed to write the configuration.
<code>version</code>	Optional	<code>["v1", "v2"]</code>	- If version is not given, it considers v1 as default argument. - If version input is v1, SDK makes API call to <code>write_configuration v1</code> API, which accepts WriteConfigModel on page 63. - If version input is v2, SDK makes API call to <code>write_configuration v2</code> API, which accepts WriteConfigModelV2 on page 65.

Table 57: Returns

Status	Description	Type	Result
Successful	Using <code>write_configuration</code> to modify CM information of given Central Unit (CU) or Distributed Unit (DU) device.	dictionary	Response <pre>{ Partial CM information of CU or DU device}</pre>

Status	Description	Type	Result
Unsuccessful	Failure in modify CM information of CU or DU device	Exceptions on page 70	Method raises an instance of exception if the operation is unsuccessful.

Query Alarm Info

Method

```
query_alarm_info(self, queryAlarmInfoDetails)
```

Description

This method allows rApp to fetch the current alarm data of the given network device from the Laguna datastore.

This method takes a dictionary as an input, a sample input can be found in the following table.

Table 58: Input Parameters

Key	Mandatory/Optional	Type	Description
<code>queryAlarmInfoDetails</code>	Mandatory	AlarmDataRequestModel on page 67	Dictionary containing the query details needed for fetching the information from the CME.

Table 59: Returns

Status	Description	Type	Result
Successful	Alarm data was successfully queried from the datastore.	list[dictionary]	List of dictionary or dictionaries where each dictionary is an alarm data record.
Unsuccessful	Failure in query alarm data from datastore.	Exceptions on page 70	Method raises an instance of exception if the operation is unsuccessful.

Query Performance Info

Method

```
query_performance_info(self, queryPerformanceInfoDetails)
```

Description

This method allows rApp to fetch the current performance data of the given network device from the Laguna datastore.

This method takes a dictionary as an input, a sample input can be found in the following table.

Table 60: Input Parameters

Key	Mandatory/Optional	Type	Description
<code>queryPerformanceInfoDetails</code>	Mandatory	PmDataRequestModel on page 68	Dictionary containing the query details needed for fetching the information from the CME.

Table 61: Returns

Status	Description	Type	Result
Successful	Performance metric data was successfully queried from the datastore.	list[dictionary]	List of dictionaries, where each dictionary is a performance metric data record.
Unsuccessful	Failure in query performance metric data from the datastore.	Exceptions on page 70	Method raises an instance of exception if the operation is unsuccessful.

A1 Related Services (A1 Termination) Package

The following methods of the SDK support APIs exposed by the A1 termination.



Note: Before attempting to call the following methods, the A1 client class must be initialized, which can be done by adding the following codes in the Python file.

Command

```
from rappsdk.a1_termination.a1_client import A1Client
a1_client_instance = A1Client(a1_url, username, password)
```

Table 62: Parameters

Key	Mandatory/Optional	Type	Description
<code>a1_url</code>	Mandatory	string	A1 server URL
<code>username</code>	Mandatory	string	Username registered in keycloak
<code>password</code>	Mandatory	string	Password

- [Query Policy Identifiers](#) on page 39
- [Query Policies](#) on page 39
- [Create Policies](#) on page 40
- [Update Policies](#) on page 42
- [Query Policy Status](#) on page 43
- [Subscribe To Policy Status Change](#) on page 44
- [Delete Policy](#) on page 45
- [Query Policy Type Identifiers](#) on page 46

- [Subscribe To Policy Type Availability Change](#) on page 46
- [Query Policy Type](#) on page 47
- [Query NearRT-RIC Identifiers](#) on page 48

Query Policy Identifiers

Method

```
query_all_policy_identifiers(self, rAppId, nearRtRicId, policyTypeId)
```

Description

This method allows rApps to fetch policy IDs for one of the `policyTypes` on a NearRT-RIC; if the `policyType` is given as input, if it is not given, the policy IDs corresponding to all of the `policyTypes` on that NearRT-RIC would be fetched and would return as an array.

Table 63: Input Parameters

Key	Mandatory/Optional	Type	Description
<code>rAppId</code>	Mandatory	string	rApp identifier
<code>nearRtRicId</code>	Mandatory	string	NearRT-RIC identifier
<code>policyTypeId</code>	Optional	string	Policy type identifier

Table 64: Returns

Status	Description	Type	Result
Successful	List of policy identifiers	list	Response <pre>[<List of Policy Ids>]</pre>
Unsuccessful	Failure in query performance metric data from the datastore	Exceptions on page 70	Method raises an instance of exception if the operation is unsuccessful.

Query Policies

Method

```
get_all_policy(self, rAppId, nearRtRicId, policyIds)
```

Description

This method allows rApps to fetch policy objects for one or more policies if specified in the input, or else get policy objects for all the policies created on the NearRT-RIC given in the input. This method accepts `rAppId`, `nearRtRicId`, and an instance of `PolicyIds` class as an input and returns list of dictionaries with policy details, for example, `policyId` and `policyObject`.

Table 65: Input Parameters

Key	Mandatory/Optional	Type	Description
<code>rAppId</code>	Mandatory	string	rApp identifier
<code>nearRtRicId</code>	Mandatory	string	NearRT-RIC identifier
<code>policyIds</code>	Optional	PolicyIds on page 69	Dictionary containing the policy identifiers whose policy object has to be fetched.

Table 66: Returns

Status	Description	Type	Result
Successful	Success in querying policies	list[dictionary]	Response <pre>[{ "policy_id": "string", "policy_object": Dictionary }]</pre>
Unsuccessful	Failure in querying policies	Exceptions on page 70	Method raises an instance of exception if the operation is unsuccessful.

Table 67: Parameters

Key	Type	Description
<code>policy_id</code>	string	ID of the policy.
<code>policy_object</code>	dictionary	Complete policy object associated with the mentioned <code>policy_id</code> .

Create Policies

Method

```
create_policies(self, rAppId, nearRtRicId, policyDetails,
notificationDestination)
```

Description

This API allows rApps to create one or more policies on the NearRT-RIC. The API takes `rAppId`, `nearRtRicId`, `notificationDestination`, and list of instances of `Policy` class and returns the policy details, for example, `policy_id`, `policy_object`, and `policy_type_id` of successfully created policies.

Table 68: Input Parameters

Key	Mandatory/ Optional	Type	Description
rAppId	Mandatory	string	rApp identifier
nearRtRicId	Mandatory	string	NearRT-RIC identifier
notificationDestination	Optional	string	Base URL of the rApp if it wants to subscribe to policy status changes. This base URL is combined with the rApp notification endpoint; for example, /policytypes/{policy_type_id}/policies/{policy_id}/statuschange to send policy status change notification to the rApp.
policyDetails	Mandatory	list[Policy on page 69]	List of policy details, for example, policy_type_id and policy_object to be created on the given NearRT-RIC.

Table 69: Returns

Status	Description	Type	Result
Successful	Success in creation of policies	list[dictionary]	Response <pre>[{ "policy_object": Dictionary, "policy_type_id": "string", "policy_id": "string" }]</pre>
Unsuccessful	Failure in creation of policies	Exceptions on page 70	Method raises an instance of exception if the operation is unsuccessful.

Table 70: Parameters

Key	Type	Description
policy_id	string	ID of the created policy.
policy_object	dictionary	Policy object created using the policy schema provided by the policy type ID.
policy_type_id	string	Id of the policy type.

Update Policies

Method

```
update_policies(self, rAppId, nearRtRicId, policyDetails,
notificationDestination)
```

Description

This API allows rApps to update one or more policies created on the NearRT-RIC. The API creates new policy on the given NearRT-RIC if a policy that rApp is trying to update is not created on the given NearRT-RIC. The API takes `rAppId`, `nearRtRicId`, `notificationDestination`, and list of instances of policy class and returns the policy details, for example, `policy_id`, `policy_object`, and `policy_type_id` of successfully upserted policies.

Table 71: Input Parameters

Key	Mandatory/Optional	Type	Description
<code>rAppId</code>	Mandatory	string	rApp identifier
<code>nearRtRicId</code>	Mandatory	string	NearRT-RIC identifier
<code>notificationDestination</code>	Optional	string	Base URL of the rApp if it wants to update the policy status change notification destination.
<code>policyDetails</code>	Mandatory	list[Policy on page 69]	List of policy details, for example, <code>policy_type_id</code> and <code>policy_object</code> to be upserted on the given NearRT-RIC.

Table 72: Returns

Status	Description	Type	Result
Successful	Success in update of policies.	list[dictionary]	Response <pre>[{ "policy_object": Dictionary, "policy_type_id": "string", "policy_id": "string" }]</pre>
Unsuccessful	Failure in update of policies.	Exceptions on page 70	Method raises an instance of exception if the operation is unsuccessful.

Table 73: Parameters

Key	Type	Description
<code>policy_id</code>	string	ID of the created policy.

Key	Type	Description
<code>policy_object</code>	dictionary	Policy object created using the policy schema provided by the policy type ID.
<code>policy_type_id</code>	string	ID of the policy type.

Query Policy Status

Method

```
get_all_policy_status(self, rAppId, nearRtRicId, policyIds)
```

Description

This method allows rApps to fetch policy status for one or more policies if specified in the input, or else get policy status for all the policies created on the NearRT-RIC given in the input. This method accepts `rAppId`, `nearRtRicId`, and an instance of `PolicyIds` class as input and returns list of dictionaries having policy status details, for example, `policy_id`, `policy_enforce_status`, and `policy_enforce_reason` (if policy status is not-enforced).

Table 74: Input Parameters

Key	Mandatory/Optional	Type	Description
<code>rAppId</code>	Mandatory	string	rApp identifier
<code>nearRtRicId</code>	Mandatory	string	NearRT-RIC identifier
<code>policyIds</code>	Optional	PolicyIds on page 69	Dictionary containing the policy identifiers whose policy status has to be fetched.

Table 75: Returns

Status	Description	Type	Result
Successful	Success in querying the policy status when policy_enforce_status is enforced.	list[dictionary]	Response <pre>[{ "policy_id": "string", "policy_enforce_status": "string" }]</pre>
	Success in querying the policy status when policy_enforce_status is not-enforced.	list[dictionary]	Response <pre>[{ "policy_id": "string", "policy_enforce_status": "string", "policy_enforce_reason": "string" }]</pre>
Unsuccessful	Failure in querying the policy status.	Exceptions on page 70	Method raises an instance of exception if the operation is unsuccessful.

Table 76: Parameters

Key	Type	Description
policy_id	string	ID of the policy
policy_enforce_status	PolicyStatusEnum on page 70	Enforce the status of the policy. It is enforced or not-enforced.
policy_enforce_reason	string	Policy enforce reason if policy status is not-enforced.

Subscribe To Policy Status Change

Method

```
subscribe_policy_status_change(self, rAppId, nearRtRicId,
policyTypeId, policyId, notificationDestination)
```

Description

This method allows an rApp to subscribe for policy status change notifications. Users must provide the details mentioned in the following input parameters to successfully receive notifications on policy status changes.

Table 77: Input Parameters

Key	Mandatory/Optional	Type	Description
rAppId	Mandatory	string	rApp identifier

Key	Mandatory/Optional	Type	Description
<code>nearRtRicId</code>	Mandatory	string	NearRT-RIC identifier
<code>policyTypeId</code>	Mandatory	string	Policy type identifier
<code>policyId</code>	Mandatory	string	Policy identifier
<code>notificationDestination</code>	Mandatory	string	rApp endpoint to receive the notification on policy status change.

Table 78: Returns

Status	Description	Type	Result
Successful	Subscribed to policy status change.	dictionary	Response <pre>{ }</pre>
Unsuccessful	Failure in subscribing to policy status change.	Exceptions on page 70	Method raises an instance of exception if the operation is unsuccessful.

Delete Policy

Method

```
delete_a1_policy(self, rAppId, nearRtRicId, policyTypeId, policyId)
```

Description

This method allows an rApp to delete a policy. Users must provide the details mentioned in the following input parameters to successfully delete.

Table 79: Input Parameters

Key	Mandatory/Optional	Type	Description
<code>rAppId</code>	Mandatory	string	rApp identifier
<code>nearRtRicId</code>	Mandatory	string	NearRT-RIC identifier
<code>policyTypeId</code>	Mandatory	string	Policy type identifier
<code>policyId</code>	Mandatory	string	Policy identifier

Table 80: Returns

Status	Description	Type	Result
Successful	Successfully deleted the policy.	null	On successful deletion, None is returned.
Unsuccessful	Failure in deleting the policy	Exceptions on page 70	Method raises an instance of exception if the operation is unsuccessful.

Query Policy Type Identifiers

Method

```
get_policy_type_ids(self, rAppId, nearRtRicId)
```

Description

This method allows rApps to fetch policy type identifiers published by the NearRT-RIC ID given in the input. The method takes rAppId and NearRT-RIC ID in the input and returns list of policy type identifiers published by the given NearRT-RIC.

Table 81: Input Parameters

Key	Mandatory/Optional	Type	Description
rAppId	Mandatory	string	rApp identifier
nearRtRicId	Mandatory	string	NearRT-RIC identifier

Table 82: Returns

Status	Description	Type	Result
Successful	Success in querying policy type identifiers that are published by the given NearRT-RIC.	list[string]	Response <pre>[<List of Policy Type Ids>]</pre>
Unsuccessful	Failure in querying policy type identifiers that are published by the given NearRT-RIC.	Exceptions on page 70	Method raises an instance of exception if the operation is unsuccessful.

Subscribe To Policy Type Availability Change

Method

```
subscribe_policy_type_availability_change(self, rAppId, nearRtRicId, policyTypeIds, notificationDestination)
```

Description

This method allows an rApp to subscribe for policy type availability change notifications. User must provide the details mentioned in the following input parameters to successfully receive notifications on changes to policy types.

Table 83: Input Parameters

Key	Mandatory/Optional	Type	Description
rAppId	Mandatory	string	rApp identifier
nearRtRicId	Mandatory	string	NearRT-RIC identifier
policyTypeIds	Mandatory	PolicyTypeIds on page 70	Policy type identifier
notificationDestination	Mandatory	string	rApp endpoint to receive notification on policy type changes.

Table 84: Returns

Status	Description	Type	Result
Successful	Subscribed to policy type availability change	dictionary	Response <pre>{ }</pre>
Unsuccessful	Failure in subscribing to policy type availability change	Exceptions on page 70	Method raises an instance of exception if the operation is unsuccessful.

Query Policy Type

Method

```
get_al_policy_type(self, rAppId, nearRtRicId, policyTypeIds)
```

Description

This method allows rApps to fetch policy type schemas for one or more policies if specified in the input, or else get policy type schemas for all the policy types published by the NearRT-RIC given in the input. This method accepts rAppId, nearRtRicId, and an instance of PolicyTypeIds class as input and returns list of dictionaries with policy type details, for example, policyId and policySchema.

Table 85: Input Parameters

Key	Mandatory/Optional	Type	Description
rAppId	Mandatory	string	rApp identifier
nearRtRicId	Mandatory	string	NearRT-RIC identifier
policyTypeId	Optional	PolicyTypeIds on page 70	Dictionary containing the policy type identifiers whose schema has to be fetched.

Table 86: Returns

Status	Description	Type	Result
Successful	Success in querying policy type	list[dictionary]	Response <pre> [{ "policy_type_id": "string", "policy_schema": Dictionary }] </pre>
Unsuccessful	Failure in querying policy type	Exceptions on page 70	Method raises an instance of exception if the operation is unsuccessful.

Table 87: Parameters

Key	Type	Description
<code>policy_type_id</code>	string	ID of the policy type.
<code>policy_schema</code>	dictionary	Complete policy object associated with the mentioned policy type ID.

Query NearRT-RIC Identifiers

Method

```
get_near_rt_ric_ids(self, rAppId)
```

Description

This method allows rApps to get list of active NearRT-RIC identifiers. This method takes `rAppId` as an input and returns list of active NearRT-RIC identifiers.

Table 88: Input Parameters

Key	Mandatory/Optional	Type	Description
<code>rAppId</code>	Mandatory	string	rApp identifier

Table 89: Returns

Status	Description	Type	Result
Successful	Success in querying NearRT-RIC identifiers	list[string]	Response <pre> [List of active nearrt_ric identifiers] </pre>
Unsuccessful	Failure in querying NearRT-RIC identifiers	Exceptions on page 70	Method raises an instance of exception if the operation is unsuccessful.

Schema

- [rAppRegister](#) on page 49
- [appTypeEnum](#) on page 50
- [serviceRegister](#) on page 50
- [httpMethodEnum](#) on page 51
- [accessMethodsEnum](#) on page 52
- [serviceUpdate](#) on page 52
- [subscribeRequest](#) on page 53
- [unSubscribeService](#) on page 54
- [dataProducerLimits](#) on page 55
- [accessMethodEnum](#) on page 55
- [pushPull](#) on page 56
- [status](#) on page 56
- [transferMethod](#) on page 56
- [subscribeDataTypeRequest](#) on page 57
- [discoverDataType](#) on page 57
- [dataTypeRegister](#) on page 58
- [subscribeDataRequest](#) on page 60
- [pullDataRequest](#) on page 61
- [pushDataRequest](#) on page 61
- [requestDataRequest](#) on page 62
- [WriteConfigModel](#) on page 63
- [ReadConfigModelV2](#) on page 64
- [ReadConfigModel](#) on page 65
- [WriteConfigModelV2](#) on page 65
- [WriteConfigUrlParamModel](#) on page 66
- [AlarmDataRequestModel](#) on page 67
- [PmDataRequestModel](#) on page 68
- [Policy](#) on page 69
- [PolicyIds](#) on page 69
- [PolicyTypeIds](#) on page 70
- [PolicyStatusEnum](#) on page 70

rAppRegister

Class Name

```
rAppRegister
```

Full Path

```
rappsdk.sme_schemas.rapp.rAppRegister
```

Description

The data model for rApp registration information.

Table 90: Parameters

Key	Mandatory/Optional	Type	Description
rAppName	Mandatory	string	Name of the rApp

Key	Mandatory/Optional	Type	Description
vendor	Mandatory	string	Name of vendor
rAppVersion	Mandatory	string	Version of rApp
certificates	Optional	dictionary	Certificates. Currently this can be empty dictionary.
appType	Mandatory	appTypeEnum on page 50	Type of the rApp.
interfaceEndpoint	Mandatory	string	URL endpoint of rApp where it is deployed.
webUILink	Optional	string	Web UI link of the rApp.



Note: Instance or object of this class is input to [Register rApp](#) on page 6.

appTypeEnum

Class Name

appTypeEnum

Full Path

rappsdk.sme_schemas.enums.appTypeEnum

Description

The enumeration class to indicate the app types supported for rApp.

Table 91: Member

Value	Description
rapp	Indicates that app type is rApp.
plugin	Indicates that app type is plugin.
internal	Indicates that app type is internal.

serviceRegister

Class Name

serviceRegister

Full Path

rappsdk.sme_schemas.service.serviceRegister

Description

The data model for service registration information.

Table 92: Parameters

Key	Mandatory/Optional	Type	Description
rAppId	Mandatory	string	Id of the rApp.
serviceName	Mandatory	string	Name of the service.
serviceVersion	Mandatory	string	Version of the service.
accessMethod	Mandatory	accessMethodsEnum on page 52	accessMethod for the given service
httpMethod	Optional	httpMethodEnum on page 51	HTTP method to invoke the provided serviceEndpoint.
communicationMechanism	Mandatory	dictionary	Communication mechanism. Currently this can be empty dictionary.
serviceEndpoint	Mandatory	string	URL or endpoint of the service.
serviceLimitations	Mandatory	dictionary	Limitations of the service. Currently this can be empty dictionary.
serviceCapabilities	Mandatory	dictionary	Capabilities of the service. Currently this can be empty dictionary.



Note: Instance or object of this class is input to [Service Registration](#) on page 11.

httpMethodEnum

Class Name

httpMethodEnum

Full Path

rappsdk.sme_schemas.enums.httpMethodEnum

Description

The httpMethodEnum is Enumeration class to indicate the HTTP methods for the services registered in SME.

Table 93: Member

Value	Description
POST	Indicates that HTTP method for the service is POST.
PUT	Indicates that HTTP method for the service is PUT.
GET	Indicates that HTTP method for the service is GET.
DELETE	Indicates that HTTP method for the service is DELETE.
PATCH	Indicates that HTTP method for the service is PATCH.

accessMethodsEnum

Class Name

accessMethodsEnum

Full Path

rappsdk.sme_schemas.enums.accessMethodsEnum

Description

This is Enumeration class to indicate the access types for the services and data types.

Table 94: Member

Value	Description
MsgBus	Indicates that MsgBus is used for communication.
RESTApi	Indicates that RESTApi is used for communication.

serviceUpdate

Class Name

serviceUpdate

Full Path

rappsdk.sme_schemas.service.serviceUpdate

Description

The data model for updating service details.

Table 95: Parameters

Key	Mandatory/Optional	Type	Description
serviceVersion	Optional	string	Version of the service.
accessMethod	Optional	accessMethodsEnum on page 52	Indicates the access type for the given service.
communicationMechanism	Optional	dictionary	Indicates the communication mechanism for the given service or data type.
serviceEndpoint	Optional	string	URL of endpoint of the service.
httpMethod	Optional	httpMethodEnum on page 51	HTTP method to invoke the provided serviceendpoint.
serviceLimitations	Optional	dictionary	Limitations of the service.
serviceCapabilities	Optional	dictionary	Capabilities of the service



Note: Instance or object of this class is input to [Service Update](#) on page 15.

subscribeRequest

Class Name

subscribeRequest

Full Path

rappsdk.sme_schemas.subscribe.subscribeRequest

Description

This data model for subscribe service information.

Table 96: Parameters

Key	Mandatory/Optional	Type	Description
rAppId	Mandatory	string	ID of the rApp which wishes to subscribe.

Key	Mandatory/Optional	Type	Description
subscribedServices	Mandatory	list[dict]	List of dictionaries containing information of serviceName and serviceVersion to be subscribed. Example <pre>[{ "serviceName": "string", "serviceVersion": "string" }]</pre>
serviceName	Mandatory	string	Name of the service.
serviceVersion	Optional	string	Version of the service. If version is not given, the default Version 1.0 is considered.



Note: Instance or object of this class is input to [Subscribe Service](#) on page 19.

unSubscribeService

Class Name

unSubscribeService

Full Path

rappsdk.sme_schemas.subscribe.unSubscribeService

Description

The data model for unsubscribe service information.

Table 97: Parameters

Key	Mandatory/Optional	Type	Description
rAppId	Mandatory	string	ID of the rApp which wishes to unsubscribe.
subscribedServices	Mandatory	list[dict]	List of dictionaries containing information of serviceName and serviceVersion to be unsubscribed. Example <pre>[{ "serviceName": "string", "serviceVersion": "string" }]</pre>
serviceName	Mandatory	string	Name of the service.

Key	Mandatory/Optional	Type	Description
<code>serviceVersion</code>	Optional	string	Version of the service. If version is not given, the default Version 1.0 is considered.



Note: Instance or object of this class is input to [Unsubscribe Service](#) on page 20.

dataProducerLimits

Class Name

`dataProducerLimits`

Full Path

`rappsdk.dme_schemas.data_type.dataProducerLimits`

Description

Data model to set limits for data producer.

Table 98: Parameters

Key	Mandatory/Optional	Type	Value	Description
<code>maxPushTime</code>	Mandatory	integer	1... 2147483647	Can be set to 0 (zero) if value is not known.
<code>maxConsumers</code>	Mandatory	integer	1... 2147483647	Number of maximum consumers rApp can serve.
<code>maxTransferRate</code>	Mandatory	integer	1... 2147483647	Can be set to 0 (zero) if value is not known.

accessMethodEnum

Class Name

`accessMethodsEnum`

Full Path

`rappsdk.dme_schemas.enums.accessMethodsEnum`

Description

This is enumeration class to indicate the access types for the services and data types.

Table 99: Member

Value	Description
<code>MsgBus</code>	Indicates that MsgBus is used for communication.
<code>RESTApi</code>	Indicates that RESTApi is used for communication.

pushPull

Class Name

pushPull

Full Path

rappsdk.dme_schemas.enums.pushPull

Description

This is enumeration class to indicate the collection method type for the data transfer.

Table 100: Member

Value	Description
pull	Pull is the producer desired collection method. It means, the new data instance is provided to DME using producer <code>pull_data()</code> API.
push	Push is the producer desired collection method. It means, the new data instance (corresponding to the data type) can be sent directly to DME using DME <code>push_data()</code> API.

status

Class Name

status

Full Path

rappsdk.dme_schemas.enums.status

Description

This is enumeration class to indicate the rApp status.

Table 101: Member

Value	Description
active	Corresponds to value <code>ACTIVE</code> , which indicates that rApp is active.
inactive	Corresponds to value <code>INACTIVE</code> , which indicates that rApp is inactive.

transferMethod

Class Name

transferMethod

Full Path

rappsdk.dme_schemas.enums.transferMethod

Description

This is enumeration class to indicate the transfer method.

Table 102: Member

Value	Description
transferMethodA	Corresponds to value TRANSFER_METHOD_A.
transferMethodB	Corresponds to value TRANSFER_METHOD_B.

subscribeDataTypeRequest

Class Name

subscribeDataTypeRequest

Full Path

rappsdk.dme_schemas.data_type.subscribeDataTypeRequest

Description

Data model for subscribe data type.

Table 103: Parameters

Key	Mandatory/Optional	Type	Description
rappId	Mandatory	string	Unique identifier of rApp which is generated and returned by SME when registering rApp.
interfaceEndpoint	Mandatory	string	interfaceEndpoint has to be in valid URL format. For example: http://rapp.<ip address>.nip.io/. An external rApp or microservice communicates with this rApp through this URL (API communication). It can be empty string if the value is not known.
dataTypeId	Mandatory	string	dataTypeId can be any unique alphanumeric string. It cannot be empty string.
collectionMethod	Mandatory	string or pushPull on page 56	Method to be used by rApp.
scope	Mandatory	string	—
target	Mandatory	string	—

discoverDataType

Class Name

discoverDataType

Full Path `rappsdk.dme_schemas.data_type.discoverDataType`

Description

Data model for discovering data type.

Table 104: Parameters

Key	Mandatory/Optional	Type	Description
<code>rappId</code>	Mandatory	string	Unique identifier of rApp which is generated and returned by SME when registering rApp.
<code>producerLimitsMaxPushTime</code>	Optional	integer	—
<code>producerLimitsMaxConsumers</code>	Optional	integer	—
<code>dataSchema</code>	Optional	string	<code>dataTypeId</code> can be any unique alphanumeric string. It cannot be empty string.
<code>collectionMethod</code>	Optional	string or pushPull on page 56	Method to be used by rApp.



Note: Instance or object of this class is input to [Discover Data Type](#) on page 28.

dataTypeRegister

Class Name `dataTypeRegister`

Full Path `rappsdk.dme_schemas.data_type.dataTypeRegister`

Description

Data model for registering the data type.

Table 105: Parameters

Key	Mandatory/Optional	Type	Value	Description
<code>dataTypeId</code>	Mandatory	string	—	<code>dataTypeId</code> can be any unique alphanumeric string. It cannot be empty string.

Key	Mandatory/ Optional	Type	Value	Description
rAppId	Mandatory	string	—	Unique identifier of rApp which is generated and returned by SME when registering rApp.
accessMethod	Mandatory	string	"RESTApi", "MsgBus", "WS"	DME supports three access methods. String should be one of those three.
collectionMethod	Mandatory	string	"PUSH", "PULL"	PUSH: Push is the producer desired collection method. It means, the new data instance (corresponding to the data type) can be sent directly to DME using DME <code>push_data()</code> API. PULL: Pull is the producer desired collection method. It means, the new data instance is provided to DME using producer <code>pull_data()</code> API.
availabilityStatus	Mandatory	string	"ACTIVE", "INACTIVE"	Activity status of the data type
collectionPeriod	Mandatory	integer	1...2147483647	Input has to be an integer. It can be set to 0 (Zero) if value is not known.
dataSchema	Mandatory	list[dict]	—	Schema of the data type. Data corresponding to the data type should always respect the given schema.
producerLimits	Mandatory	dictionary	—	Dictionary which contains information related to serving limitations of producers with following structure. Note that all these fields are mandatory. If value is not known, it can be set to 0. Example <pre>{ "maxPushTime":integer, "maxConsumers":integer, "maxTransferRate":integer }</pre>

Key	Mandatory/ Optional	Type	Value	Description
producerCapabilities	Mandatory	string	"TRANSFER_METHOD_A", "TRANSFER_METHOD_B"	Input can be either TRANSFER_METHOD_A or TRANSFER_METHOD_B.



Note: Instance or object of this class is input to [Register Data Type](#) on page 23.

subscribeDataRequest

Class Name

subscribeDataRequest

Full Path

rappsdk.dme_schemas.subscribe_data.subscribeDataRequest

Description

A class of schemas for DME when subscribing to a data type.

Table 106: Parameters

Key	Mandatory/ Optional	Type	Value	Description
rappId	Mandatory	string	—	Input has to be the rApp ID of the rApp which would like to subscribe for the data type.
interfaceEndpoint	Mandatory	string	—	interfaceEndpoint has to be in valid URL format, for example, http://rapp.<ip address>.nip.io/. An external rApp or microservice communicates with this rApp through this URL (API communication). It can be empty string if the value is not known.
dataTypeId	Mandatory	string	—	ID of the data type to which the rApp would like to subscribe.
collectionMethod	Mandatory	string	—	Input can be either PUSH or PULL.
collectionPeriod	Mandatory	integer	1... 2147483647	Input has to be an integer. It can be set to 0 (Zero) if the value is not known.
scope	Mandatory	string	—	Input has to be a string. Empty string " " can be sent as input if the value is not known.

Key	Mandatory/ Optional	Type	Value	Description
target	Mandatory	String	—	Input has to be a string. Empty string " " can be sent as input if the value is not known.



Note: Instance or object of this class is input to [Subscribe Data](#) on page 29.

pullDataRequest

Class Name

```
pullDataRequest
```

Full Path

```
rappsdk.dme_schemas.pull_data.pullDataRequest
```

Description

A class of schemas for DME for pulling the data.

Table 107: Parameters

Key	Mandatory/Optional	Type	Description
dataTypeId	Mandatory	string	Data type ID that needs to be referred to pull its corresponding data instance.
laguna_ref	Mandatory	string	Reference to the data instance that is requested from or pushed to <code>dataTypeId</code> . It might refer to a single data point or bulk data.



Note: Instance or object of this class is input to [Pull Data](#) on page 32.

pushDataRequest

Class Name

```
pushDataRequest
```

Full Path

```
rappsdk.dme_schemas.push_data.pushDataRequest
```

Description

A class of schemas for DME to push data.

Table 108: Parameters

Key	Mandatory/Optional	Type	Description
dataTypeId	Mandatory	string	ID of the targeted data type

Key	Mandatory/Optional	Type	Description
data	Mandatory	union[list[dictionary], dictionary]	List of dictionaries where each dictionary is the record that has to be pushed to the dataTypeId.



Note: Instance or object of this class is input to [Push Data](#) on page 33.

requestDataRequest

Class Name

requestDataRequest

Full Path

rappsdk.dme_schemas.request_data.requestDataRequest

Description

A class of schemas for DME to request data.

Table 109: Parameters

Key	Mandatory/Optional	Type	Value	Description
dataTypeId	Mandatory	string	—	Laguna store name or RESTapi based dataTypeId name.
startTime	Mandatory	datetime/dictionary	—	startTime can take a datetime input or dictionary. If dictionary is given, it selects the column for timestamp based on key provided for dictionary; otherwise, it takes default timestamp column timestampnew.
stopTime	Mandatory	datetime/dictionary	—	stopTime can take a datetime input or dictionary. If dictionary is given it selects the column for timestamp based on key provided for dictionary; otherwise, it takes default timestamp column timestampnew.
timeInterval	Mandatory	integer	1...2147483647	(Not used)

Key	Mandatory/ Optional	Type	Value	Description
period	Mandatory	integer	1...2147483647	(Not used)
target	Optional	list[dictionary]	—	Target should be provided as a list of dictionary. Each dictionary needs to have the column name as key that is used as primary index for filtering, followed by the value that needs to be filtered based on.
scope	Optional	list[dictionary]	—	Scope is a list of dictionary. Each dictionary needs to have the column name as key that is used as secondary index for filtering, followed by the value that needs to be filtered based on.
result	Optional	list	—	Result is a list that takes the columns names that should be the only values in final data pull.



Note: Instance or object of this class is input to [Request Data](#) on page 33.

WriteConfigModel

Class Name

WriteConfigModel

Full Path

rappsdk.cme_schemas.configuration_request.WriteConfigModel

Description

Data model for write configuration.

Table 110: Parameters

Key	Mandatory/Optional	Type	Description
ManagedElementId	Mandatory	string	ID of the managed element or network device whose configuration has to be updated.
GNBDUFunctionId	Mandatory	string	Unique identifier used to identify gNB-DU uniquely within gNB-CU.

Key	Mandatory/Optional	Type	Description
NRCellDUId	Mandatory	string	ID of the cell attached to the given DU whose configurations has to be updated.
ManagedNFService	Optional	string	ID of ManagedNFService under NRCellDU whose configuration has to be updated.
data	Mandatory	dictionary	JSON containing the ID and configuration parameter attributes to be updated within the managed network function list.



Note: Instance or object of this class is input to [Write Configuration](#) on page 36.

ReadConfigModelV2

Class Name

ReadConfigModelV2

Full Path

rappsdk.cme_schemas.confiiuration_request.ReadConfigModelV2

Description

Data model for read configuration v2.

Table 111: Parameters

Key	Mandatory/ Optional/ Conditional	Type	Value	Description
deviceType	Mandatory	string	["CU", "DU"]	Type of the device to which the user wish to get the CM information.
ManagedElementId	Mandatory	string	—	ID of the managed element or network device whose configuration has to be fetched.
GNBFunctionType	Optional	string	["GNBCUUPFunction", "GNBCUCPFunction", "GNBDUFunction"]	GNB function type of the managed element mentioned in ManagedElementId.

Key	Mandatory/ Optional/ Conditional	Type	Value	Description
GNBFunctionId	Optional	string	—	ID of the selected GNBFunctionType.
NRCellId	Optional	string	—	ID of the NR Cell. If device type is CU, the input is mapped to NRCellCU. If it is DU, the it is mapped to NRCellDU.



Note: Instance or object of this class is input to [Read Configuration](#) on page 35.

ReadConfigModel

Class Name

ReadConfigModel

Full Path

rappsdk.cme_schemas.configuration_request.ReadConfigModel

Description

Data model for read configuration.

Table 112: Parameters

Key	Mandatory/Optional	Type	Description
ManagedElementId	Optional	string	ID of the managed element or network device whose configuration has to be fetched.
GNBDUFunctionId	Optional	string	Unique identifier used to identify gNB-DU uniquely within gNB-CU.
NRCellDUId	Optional	string	ID of the cell attached to the given DU whose configurations has to be fetched.



Note: Instance or object of this class is input to [Read Configuration](#) on page 35.

WriteConfigModelV2

Class Name

WriteConfigModelV2

Full Path

rappsdk.cme_schemas.confiiuration_request.WriteConfigModelV2

Description

Data model for write configuration v2.

Table 113: Parameters

Key	Mandatory/ Optional/ Conditional	Type	Description
url_params	Mandatory	WriteConfigUrlParamModel on page 66	Details of the managed element, GNBFunctionType, NRCell, and ManagedNFService whose configuration has to be updated.
data	Mandatory	dictionary	JSON containing the ID and configuration parameter attributes to be updated within the managed network function list.



Note: Instance or object of this class is input to [Write Configuration](#) on page 36.

WriteConfigUrlParamModel

Class Name

WriteConfigUrlParamModel

Full Path

rappsdk.cme_schemas.confiiuration_request.WriteConfigUrlParamModel

Description

Data model for URL parameters in WriteConfigModelV2.

Key	Mandatory /Optional/ Conditiona l	Type	Range	Description
deviceType	Mandatory	string	["CU", "DU"]	Device type of the managed element to which the user wish to update the configuration.
ManagedElementId	Mandatory	string	—	ID of the managed element or network device whose configuration has to be updated.
GNBFunctionType	Mandatory	string	["GNBCUUPFunction", "GNBCUCPF unction", "GNBDUFunction"]	GNB function type of the managed element mentioned in ManagedElementId.

Key	Mandatory /Optional/ Conditional	Type	Range	Description
GNBFunctionId	Mandatory	string	—	ID of the selected GNBFunctionType.
NRCellId	Mandatory	string	—	ID of the NR Cell. If deviceType is CU, the input is mapped to NRCellCU. If it is DU, it is mapped to NRCellDU.
ManagedNFSERVICE	Conditional	string	—	ID of ManagedNFSERVICE under NRCellDU whose configuration has to be updated. To be given only when the ManagedNFSERVICE has to be updated.

AlarmDataRequestModel

Class Name

AlarmDataRequestModel

Full Path

rappsdk.cme_schemas.data_request.AlarmDataRequestModel

Description

Data model for query alarm information.

Table 114: Parameters

Key	Mandatory/Optional	Type	Description
dataType	Mandatory	string	DataType ID of the data to be fetched. The dataTypeId can be any alphanumeric string. It cannot be empty string.
target	Mandatory	list[dictionary]	Accepts list of dictionaries indicating primary filter for the data searched for in the database, and only the data that has the matching key-value pair in it is returned. Wildcard (*) values are supported.

Key	Mandatory/Optional	Type	Description
scope	Mandatory	list[dictionary]	Accepts list of dictionaries indicating secondary filters for the data. All the data having any of these dictionaries mentioned in the scope is returned.
result	Mandatory	list[string]	Accepts the parameters or fields from the data which has to be returned.
timeIndex	Mandatory	string	Index for the time field on which the time range filter has to be applied.
period	Optional	integer	Time interval between the current time and start time for data consumption. If not given or a value less than the period threshold is provided, the period threshold is considered.



Note: Instance or object of this class is input to [Query Alarm Info](#) on page 37.

PmDataRequestModel

Class Name

PmDataRequestModel

Full Path

rappsdk.cme_schemas.data_request.PmDataRequestModel

Description

Data model for query performance information.

Table 115: Parameters

Key	Mandatory/Optional	Type	Description
dataType	Mandatory	string	ID of data type to be fetched. The dataTypeId can be any alphanumeric string. It cannot be empty string.
target	Mandatory	list[dictionary]	Accepts list of dictionaries indicating primary filter for the data searched for in the database, and only the data that has the matching key-value pair in it is returned. Wildcard (*) values are supported.
scope	Mandatory	list[dictionary]	Accepts list of dictionaries indicating secondary filters for the data. All the data having any of these dictionaries mentioned in the scope is returned.
result	Mandatory	list[string]	Accepts the parameters or fields from the data which has to be returned.

Key	Mandatory/Optional	Type	Description
<code>timeIndex</code>	Mandatory	string	Index for the time field on which the time range filter has to be applied.
<code>period</code>	Optional	integer	Time interval between the current time and start time for data consumption. If not given or a value less than the period threshold is provided, the period threshold is considered.



Note: Instance or object of this class is input to [Query Alarm Info](#) on page 37.

Policy

Class Name

Policy

Full Path

rappsdk.al_termination_schemas.policy

Description

Data model for policy creation or updating.

Table 116: Parameters

Key	Mandatory/Optional	Type	Description
<code>policy_type_id</code>	Mandatory	string	Policy type ID whose corresponding policy instance has to be created or updated.
<code>policy_object</code>	Mandatory	dictionary	Policy object of the policy.

PolicyIds

Class Name

PolicyIds

Full Path

rappsdk.al_termination_schemas.policy

Description

Data model for query policy or query policy status.

Table 117: Parameters

Key	Mandatory/Optional	Type	Description
<code>policy_id</code>	Optional	list	List of policy IDs whose policy details, for example, policy object or policy status have to be fetched.

PolicyTypeIds

Class Name

PolicyTypeIds

Full Path

rappsdk.al_termination_schemas.policyType

Description

Data model for query policy type schema.

Table 118: Parameters

Key	Mandatory/Optional	Type	Description
policy_type_id	Optional	list	List of policy type IDs whose policy type details, for example, policy schema have to be fetched.

PolicyStatusEnum

Description

Indicates the possible values for policy status.

Table 119: Member

Value	Description
enforced	Indicates that the policy status is enforced.
not-enforced	Indicates that the policy status is not-enforced.

Exceptions

This section describes the exceptions that could occur from SME, DME, or CME API calls that are invoked by the SDK functions.

Table 120: Exceptions

Errors	Description
Authentication	Authentication error is raised if authorization fails on the server side when invoking the API.
Authorization	Authorization error is raised if the given user credentials are not authorized for accessing the invoked API.
Internal Server	Internal server error is raised if the server is down or is not able to process the request.

Errors	Description
Validation	Validation error is raised if the server is not able to process the request because of semantic errors in request.
Duplicate	Duplicate error is raised if the request was already made and user is trying to send the same request again.

Glossary

This chapter describes the product glossary terms and definitions.

API	Application Programming Interface
CME	Configuration Management and Exposure
CU	Central Unit
DME	Data Management and Exposure
DU	Distributed Unit
LCM	Life Cycle Management
NF	Network Function
rApp	NonRT-RIC Application
SDK	Software Development Kit
SME	Service Management and Exposure
SMO	Service Management and Orchestrator
UI	User Interface
URL	Uniform Resource Locator
Virtuora AX	Virtuora Analytics Transformation

Index

A

A1
 termination [38](#)
[accessMethodEnum](#) [55](#)
[accessMethodsEnum](#) [52](#)
[AlarmDataRequestModel](#) [67](#)
[appTypeEnum](#) [50](#)

C

[CME](#) [34](#)
 create
 A1 policy [40](#)

D

[dataProducerLimits](#) [55](#)
[dataTypeRegister](#) [58](#)
 delete policy [45](#)
 deregister [7](#), [27](#)
 discover data type [28](#)
[discoverDataType](#) [57](#)
[DME](#) [22](#)

E

exception [70](#)

H

[httpMethodEnum](#) [51](#)

I

install python SDK library [4](#)

O

overview [4](#)

P

[PmDataRequestModel](#) [68](#)
 policy [69](#)
 policyIds [69](#)
[policyStatusEnum](#) [70](#)
[policyTypeIds](#) [70](#)
 pull data [32](#)
[pullDataRequest](#) [61](#)
 push data [33](#)
[pushDataRequest](#) [61](#)
[pushPull](#) [56](#)

Q

query [37](#), [39](#)
 NearRT-RIC identifier [48](#)
 performance info [37](#)
 policy [43](#), [47](#)
 type [46](#)

R

rApp software development kit [5](#)
[rAppRegister](#) [49](#)
 read
 configuration [35](#)
[ReadConfigModel](#) [65](#)
[ReadConfigModelv2](#) [64](#)
 register [6](#), [23](#)
 request data [33](#)
[requestDataRequest](#) [62](#)

S

schema [49](#)
 service [15](#), [17](#)
 deregistration [13](#)
 registration [11](#)
[serviceRegister](#) [50](#)
[serviceUpdate](#) [52](#)
[SME](#) [5](#)
 status [56](#)
 subscribe [29](#)
 data type change [27](#)
 service [19](#)
 to policy status [44](#)
 to policy type availability change [46](#)
[subscribeDataRequest](#) [60](#)
[subscribeDataTypeRequest](#) [57](#)
[subscribeRequest](#) [53](#)

T

[transferMethod](#) [56](#)

U

unsubscribe
 data type change [28](#)
 service [20](#)
[unSubscribeService](#) [54](#)
 update
 policy [42](#)
 web UI link of rApp [9](#)

W

write [36](#)
[WriteConfigModel](#) [63](#)

WriteConfigModelv2 [65](#)

WriteConfigUrlParamModel [66](#)